

AlexNet

: ImageNet Classification with Deep Convolutional Neural
Networks

Abstract

- imageNet LSVRC-2010(1000개 클래스) 1.2million의 고해상도 이미지 분류하는 크고 깊은 convolutional network를 훈련
- test data 성능

top-1 → 37.5% error rate

top-5 → 17.0% error rate

- train 속도 빠르게 하기 (non-saturating neurons, 효율적인 GPU implementation)
- overfitting 줄이기 (data augmentation, fully-connected layer에 dropout regularization 방법 사용)
- ILSVRC-2012 competition -> 15.5% Top-5 test error rate

Introduction

- 다양한 **object** 인지 위해서는 큰 **training set**이 필요함

현재 수백만 장의 **labeled image** 수집 가능해짐

(LabelME, ImageNet)

- **large learning capacity**를 지닌 모델이 필요함

CNN -> 이미지 특성에 대한 정확한 **assumption** 만듦

통계의 정적성 (**weight sharing** 사용), 픽셀 간 국소적 의존성 (작은 크기 필터 사용)

CNN

- 같은 크기의 **standard neural network**와 비교시

더 적은 connection, parameter -> train 쉬우나 이론적으로 **best** 성능 낮을 수 있음

- **과거 CNN**

고해상도 이미지 적용에는 비용이 너무 많이 들었음

- **현재 CNN**

GPU + 최적화된 2D convolution -> 매우 큰 CNN도 오버피팅 없이 훈련 가능

=> ImageNet에 대한 가장 큰 CNN 중 하나를 훈련시켜 가장 좋은 성능을 냄

The Dataset

- **ImageNet**

22,000개의 카테고리를 가진 15 million labeled 고해상도 이미지

- **ImageNet Large-Scale Visual Recognition Challenge (ILSVRC)**

ImageNet 내 1000개 카테고리당 1000개의 이미지 사용

- **사용 버전** : ILSVRC-2010 (test label 이용가능), ILSVRC-2012 (“”이용 불가)
- **error rate** : top-1, top-5 사용

The Dataset

- 문제 : ImageNet은 다양한 고해상도 이미지로 구성되어있으나 우리는 고정된 input 차원을 원함

해결 : 이미지를 고정된 해상도인 **256x256**으로 다운 샘플링함

ex) 직사각형 이미지 -> 짧은 면을 **256** 길이로 맞추고 중앙에서 **256x256**으로 잘라냄

- 전처리

훈련셋 평균값만 각 픽셀에다 빼줌

-> 모든 이미지의 같은 위치의 픽셀이 0중심으로 조정

-> raw RGB에서 학습

3. The Architecture

- 8개의 학습된 layer

5개의 convolutional layers

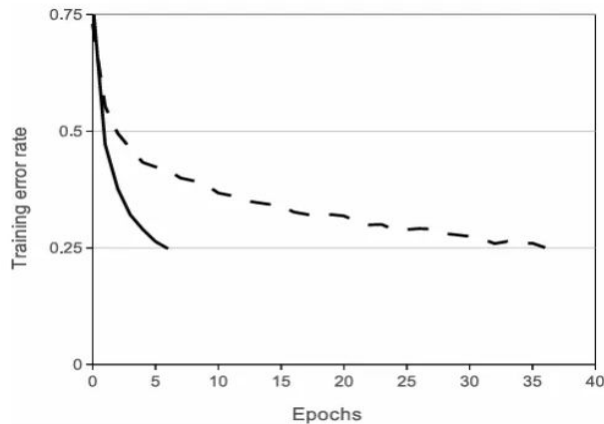
3개의 fully-connected layers

3.1 ReLU Nonlinearity

- non-saturating nonlinearity f : ReLU

$$f(x) : \max(0, x)$$

- ReLU vs tanh (in four-layer convolutional network) -> ReLU가 몇 배 더 빠름



+ Jarrett의 $f(x) = |\tanh(x)|$

- Caltech-101 dataset에서 contrast normalization + local average pooling
사용할 때 잘 작동했음

-> 오버피팅을 막는 것이 주요 문제였음, ReLU를 가지고 만들어낼 효과와는
다름

ReLU -> 빠른 학습

3.2 Training on Multiple GPUs

- 문제

single GTX 580 GPU : 오직 3GB 메모리 가짐

1.2 million training 예제 -> 모델 크기 증가 -> 단일 GPU가 감당할 수 없음!

해결

cross-GPU parallelization

2개의 GPU로 나눔, 커널을 각 gpu에 절반씩 배치 (WHY? 데이터를 만나누고)

한 GPU가 읽고 다른 GPU 메모리에 직접 씀

3.2 Training on Multiple GPUs

- 커널 간 연결구조 패턴

- > gpu는 특정 layer에서만 상호작용함

- GPU 전부 다 연결

- : layer3의 커널 <- layer2의 모든 커널맵으로 부터 input 취함

- 각각의 GPU 끼리 연결

- : layer4의 커널 <- 같은 gpu에 위치한 layer3의 특정 커널맵으로 부터 input을 취함

- 패턴 선택 -> cross-validation의 문제

- => 계산 가능 비율 되도록 통신량 조절하면서 계속 실험

3.2 Training on Multiple GPUs

- 최종 구조

columnar CNN과 유사하나 완전히 독립적이진 않음

- **cross -GPU parallelization** 적용 효과 (<-> 단일 GPU로 half 커널 사용)

top-1 error rate : 1.7%만큼 낮춤

top-5 error rate : 1.2%만큼 낮춤

train 시간도 약간 낮춤

3.3 Local Response Normalization

- **ReLU**

saturating을 막기 위한 input normalization은 필요 없음

오버피팅을 막기 위한 local normalization 진행

3.3 Local Response Normalization

- response -normalized activity

$$b_{x,y}^i = a_{x,y}^i / \left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a_{x,y}^j)^2 \right)^{\beta}$$

- a_{xy}^i : (x,y) 위치에서 커널 i를 적용한 후 ReLU nonlinearity 거쳐 뉴런에서 계산된 activity
- j: i주변의 커널 인덱스
- n: 인접 커널 수 (논문에서는 5 사용)
- N: 전체 커널 수
- k, α, β : val set을 사용하여 각각 다른 커널로부터 결정된 하이퍼파라미터
 - 최적
 - $k = 2, n = 5, \alpha = 10^{-4}, \beta = 0.75$

3.3 Local Response Normalization

=> 같은 위치에서 계산된 여러 커널의 출력값들 중 인접한 커널 n 개를 골라 제곱합 구하여 나눔

- **lateral inhibition**의 형태 지님

: 하나의 뉴런이 활성화되면 주변 뉴런의 활동을 억제하는 현상

(<- 생물학적 뉴런에서 영감받음)

=> 다른 커널에서 계산된 뉴런 사이에서 큰 **activity**들의 경쟁을 만들어냄

3.3 Local Response Normalization

- **LRN 사용 시 효과**

top-1 error rate : 1.4% 낮춤

top-5 error rate : 1.2% 낮춤

- **CIFAR-10 dataset에서의 효과**

normalization X : 13% test error rate

normalization O : 11% test error rate

3.4 Overlapping Pooling

- **Pooling layer in CNN**

같은 커널 맵에서 이웃하는 출력을 요약함

- **pooling layer**

neighborhood size : $z \times z$

각 유닛 간 간격 : s

3.4 Overlapping Pooling

- 방식

$s = z$ -> 전통적인 local pooling

$s < z$ -> 겹치는 pooling, 우리는 이것을 사용 ($s=2, z=3$)

- **Overlapping pooling 효과** ($s=2, z=2$ 와 비교 시)

top-1 error rate : 0.4% 낮춤

top-5 error rate : 0.3% 낮춤

오버피팅 문제가 완화됨 (WHY? 같은 픽셀 여러 방식 요약 -> 많은 spatial 정보
-> 다양??)

3.5 Overall Architecture

- **8 layer**로 구성

5 convolutional

3 fully connected

최종 fc layer : 1000가지 softmax (1000 class 분류)

- **학습**

multinomial logistic regression 목적함수를 최대화 함

모델이 예측한 확률 분포에서, 각 훈련 샘플의 정답 레이블에 해당하는 로그 확률을 평균 내어 최대화

3.5 Overall Architecture

- 연결 구조

2,4,5번째 convolutional layer의 커널 : 오직 같은 GPU 이전 layer 커널 맵과 연결됨

3번째 convolutional layer의 커널 : 2번째 모든 커널 맵과 연결됨

response-normalization layers : 1,2번째 layer 뒤에 위치함

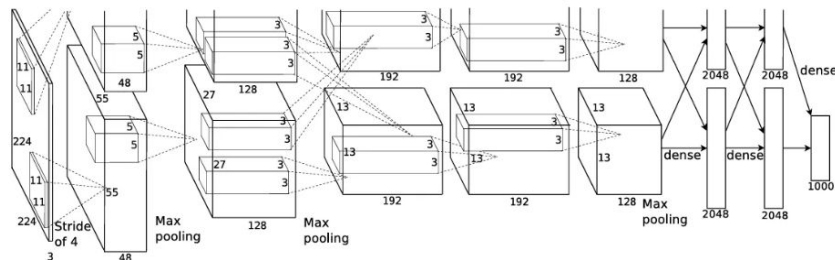
max pooling layer : 두 response-normalization layer와 5번째 convolutional layer 뒤에 위치

ReLU non-linearity : 모든 convolutional, fully connected에 적용됨

3.5 Overall Architecture

Layer	Type	Kernel / Units	#Kernels	Stride	Padding
Conv1	Convolution	11×11×3	96	4	0
LRN1	Normalization	—	—	—	—
Pool1	Max-pooling	3×3	—	2	0
Conv2	Convolution	5×5×48	256	1	2
LRN2	Normalization	—	—	—	—
Pool2	Max-pooling	3×3	—	2	0
Conv3	Convolution	3×3×256	384	1	1
Conv4	Convolution	3×3×192	384	1	1
Conv5	Convolution	3×3×192	256	1	1
Pool3	Max-pooling	3×3	—	2	0
FC6	Fully-connected	—	4096	—	—
FC7	Fully-connected	—	4096	—	—
FC8	Fully-connected	—	1000	—	—

- input : 150,528 차원
- 뉴런의 수 :
253,440—186,624—64,896—64,896—43,
264—4096—4096—1000



4. Reducing Overfitting

- 문제

뉴럴 네트워크 구조 : 60 million 파라미터를 가짐

이미지를 라벨로 매핑 시 각 training example에 10 bit 정보량만이 존재함

=> 데이터 정보량 < 모델 용량

=> 오버 피팅이 무조건 발생함!

4.1 Data Augmentation

: 라벨을 유지하고 변형을 통해 데이터 셋을 늘림 (가장 흔하고 쉬운 방법!!)

- 약간의 계산 통해 변형된 이미지를 만듦 (디스크에 저장할 필요 X)

GPU가 이전 이미지 배치 훈련 시키는 동안 **CPU**에서 파이썬 코드로 이미지 변형

-> 실제로는 추가적 계산비용이 안들

4.1 Data Augmentation

1. image translation + horizontal reflection (공간적)

랜덤으로 224x224패치를 256x256 이미지에서 추출하고 네트워크에 훈련 시킴

가능한 위치 수 : 1089개 = $(256-224+1)^2$

좌우 반전 적용 : 약 2048개의 훈련 셋 증가!

- **test시에도 적용**
- 224x224 패치 5개 추출 + horizontal reflection (→ 10개) 으로 예측을 함
네트워크 softmax를 통해 만들어진 10 패치에서의 예측값을 평균화함

4.1 Data Augmentation

2. RGB 채널의 intensity 변형 (색상)

ImageNet training set 전체 RGB 픽셀 세트에 대한 PCA를 수행

-> RGB 픽셀 값들의 3×3 공분산 행렬

- RGB image pixel $I_{xy} = [I_{xy}^R I_{xy}^G I_{xy}^B]^T$ 에 다음 값을 더해준다

$$[\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3][\alpha_1 \lambda_1, \alpha_2 \lambda_2, \alpha_3 \lambda_3]^T$$

- P_i : RGB 픽셀 값들의 3×3 공분산 행렬의 i 번째 고유 벡터
- λ_i : RGB 픽셀 값들의 3×3 공분산 행렬의 i 번째 고유 값
- α_i : 평균 0, 표준편차 0.1인 정규분포에서 샘플링한 무작위 값
 - 특정 트레이닝 이미지의 모든 픽셀에 대해 한번만 샘플링됨
 - 그 이미지가 트레이닝에 다시 사용될 때 새로 샘플링됨

4.1 Data Augmentation

2. RGB 채널의 **intensity** 변형 (색상)

색상의 톤과 강도만 바뀌게 됨!

=> 조명의 밝기와 색상이 변하는 것에 관계없이 **natural image**의 주요 특징을 포착할 수 있게 됨!

- 효과

top-1 error-rate : 1%넘게 낮춤

4.2 Dropout

- 문제

많은 다른 모델의 예측 합하기 -> **test error** 줄이기에 효과적이나 비용이 너무 많이 듦 (며칠동안 훈련시켜야 함)

- 해결

단일 모델 내부에서 약한 앙상블 효과 흉내

dropout : 0.5의 확률로 각 **hidden** 뉴런의 출력을 0으로 만듦

dropout된 뉴런은 **forward pass**에 기여할수도 **backpropagation**에 참여할 수도
X

input이 들어가면 신경망은 서로 다른 구조를 샘플링하나 **weight**는 공유함

4.2 Dropout

- 첫번째 2개의 **fully-connected layers**에서 **dropout**을 진행함

(+) 특정 뉴런의 존재에 의존하지 않게 되어 뉴런의 복잡한 상호의존성을 낮춤

(+) 다른 뉴런의 랜덤 **subset**과 결합하여 강건한 **feature** 학습 가능

(-) 오버 피팅 문제는 완화되나, 수렴까지 필요한 반복횟수가 대략 두배로 늘어남!

- **test시**

모든 뉴런을 사용하나 **output**에 **0.5**를 곱해줌

-> 수많은 **dropout** 네트워크들에 의해 생성된 예측 분포의 기하 평균을 취한것과 같게 근사

5. Details of learning

- 학습

SGD 이용 훈련

배치 사이즈 : 128 examples

momentum : 0.9

weight decay : 0.0005

-> 작은 weight decay로 단순 regularizer이 아닌 모델의 training error를 낮출 수 있었음

5. Details of learning

- **weight w update rule**

$$v_{i+1} := 0.9 \cdot v_i - 0.0005 \cdot \epsilon \cdot w_i - \epsilon \cdot \left\langle \frac{\partial L}{\partial w} \Big|_{w_i} \right\rangle_{D_i}$$

$$w_{i+1} := w_i + v_{i+1}$$

i : iteration index

v: momentum variable

ϵ : learning rate

$\left\langle \frac{\partial L}{\partial w} \Big|_{w_i} \right\rangle_{D_i}$: 현재 파라미터 w 에서 미니 배치 D 에 대해 계산한 손실함수 **gradient**
평균

5. Details of learning

- 초기화

w : 평균이 0이고 표준 편차가 0.01인 정규분포 **zero-mean** 가우시안에서 샘플링

bias

2,4,5 convolutional + fully-connected hidden layers : 상수 1

-> ReLU에 positive input 제공 -> 초기 학습단계를 가속화함

나머지 뉴런 : 상수 0

5. Details of learning

- **learning rate**

모든 layer에 동일한 값을 사용, training 내내 수동 조정함

validation error rate가 개선되지 않으면 learning rate $\times 1/10$

초기값 0.01에서 세번 감소됨 (-> 0.00001)

- 1.2million image의 훈련 세트를 90cycle로 훈련시킴

-> NVIDIA GTX 580 3GB GPU 2개로 5~6일 이 걸렸음

6. Result

- ILSVRC-2010 성능 결과표

Model	Top-1	Top-5
<i>Sparse coding [2]</i>	47.1%	28.2%
<i>SIFT + FVs [24]</i>	45.7%	25.7%
CNN	37.5%	17.0%

6. Result

- ILSVRC-2012 competition (test set 공개 X)

Model	Top-1 (val)	Top-5 (val)	Top-5 (test)
<i>SIFT + FVs [7]</i>	—	—	26.2%
1 CNN	40.7%	18.2%	—
5 CNNs	38.1%	16.4%	16.4%
1 CNN*	39.0%	16.6%	—
7 CNNs*	36.7%	15.4%	15.3%

6. Result

- **Fall 2009 version of ImageNet with 10,184 categories and 8.9 million images**

데이터의 반을 트레이닝에 나머지 반을 테스트에 사용함

Top-1 error rate : 67.4%

Top-5 error rate : 40.9%

Best published results : 78.1%, 60.9%

6.1 Qualitative Evaluation

- 두 개의 **data-connected layer**로 학습된 **convolutional kernels**

top 48개의 커널 : GPU1, color-agnostic

나머지 커널 : GPU2, color-specific



6.1 Qualitative Evaluation

- 8개 이미지에 대한 top-5 prediction



-> 꽤나 합리적인 결과임

6.1 Qualitative Evaluation

- 유클리디안 유사도 측정

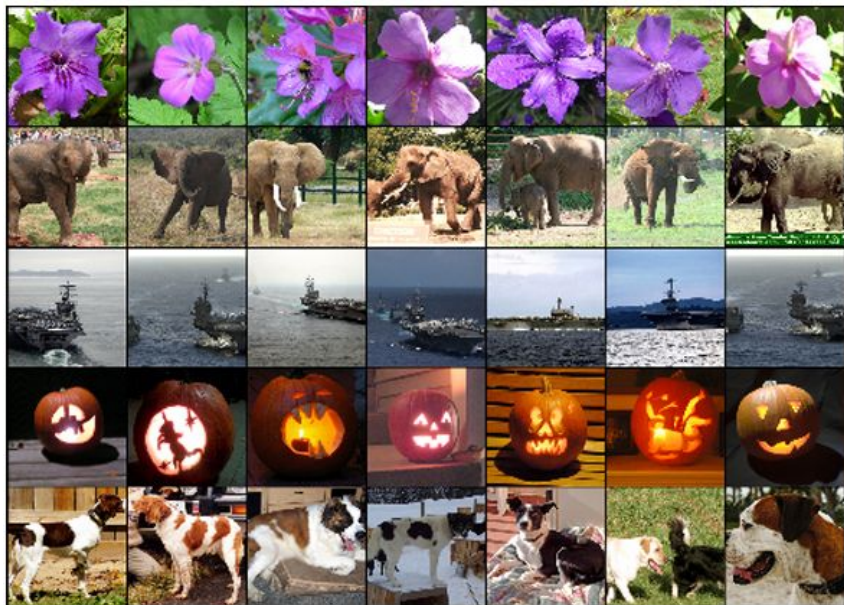
마지막 4096-차원의 히든 레이어의 한 이미지로 부터 유도된 **feature activation**

2개의 이미지 → 유클리디안 **separation**으로 **feature vector**를 만듦

-> 높은 수준의 네트워크를 그 둘을 비슷하다고 볼 것임!

=> 이미지를 숫자 벡터로 바꾼 후 벡터간 거리로 이미지 비슷함을 판단가능!

6.1 Qualitative Evaluation



- 이러한 **measure**에서 5개의 테스트 이미지와 6개의 훈련 이미지가 유사함
- 픽셀 수준에서 얻어진 훈련 이미지들은 첫번째 칼럼에 있는 쿼리 이미지와 **L2**거리 측정 시 가까움

6.1 Qualitative Evaluation

- 문제

4096 차원 real valued vector의 유클리디안 거리 유사도는 비효율적

- 해결

오토인코더로 이 벡터를 짧은 이진 코드로 압축

- **Raw픽셀 오토 인코딩**

한계 : 이미지 라벨 사용 없음, 의미적으로 비슷하던 말던 엣지 패턴의 유사도만을 가지고 이미지를 검색함

-> 빠른 검색 가능, 효율적

7. Discussion

- 크고 깊은 convolutional neural network가 매우 어려운 데이터셋에서 **단순 지도 학습**만으로 이전 기록들을 뛰어넘는 결과를 얻을 수 있었음
- **Depth**가 매우 중요한 요소였음

ex) 중간 레이어 삭제 → 2%의 top-1 성능 감소

- **발전 가능성**

충분한 연산 자원이 확보-> 라벨있는 데이터 양을 늘리지 않고도 네트워크 크기를 확장할 수 있다면 성능이 더 좋아질 것

비디오 시퀀스도 데이터로 사용 -> 시간적 정보를 활용해 더 정확한 인식 모델이 가능해질 것

END